Hosting Platforms

EMR, Databricks

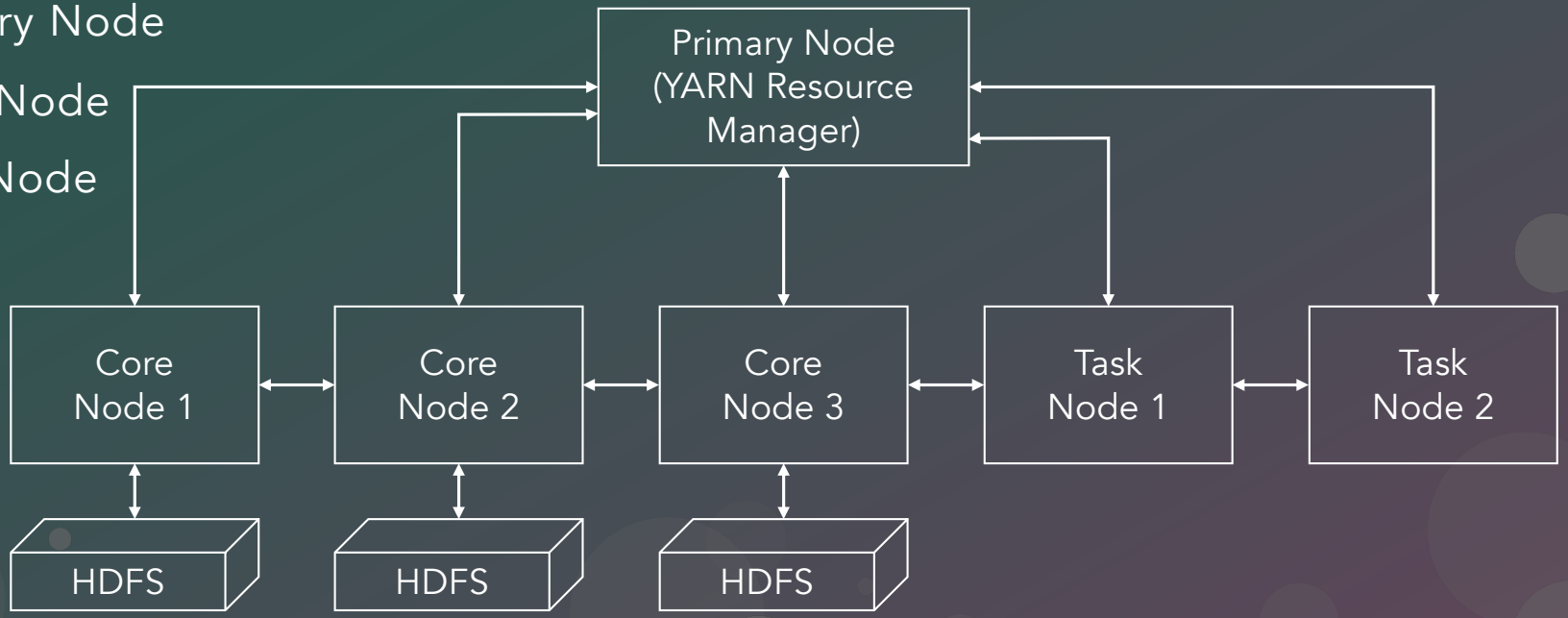
www.3aayaam.in

www.3aayaam.in

AWS EMR

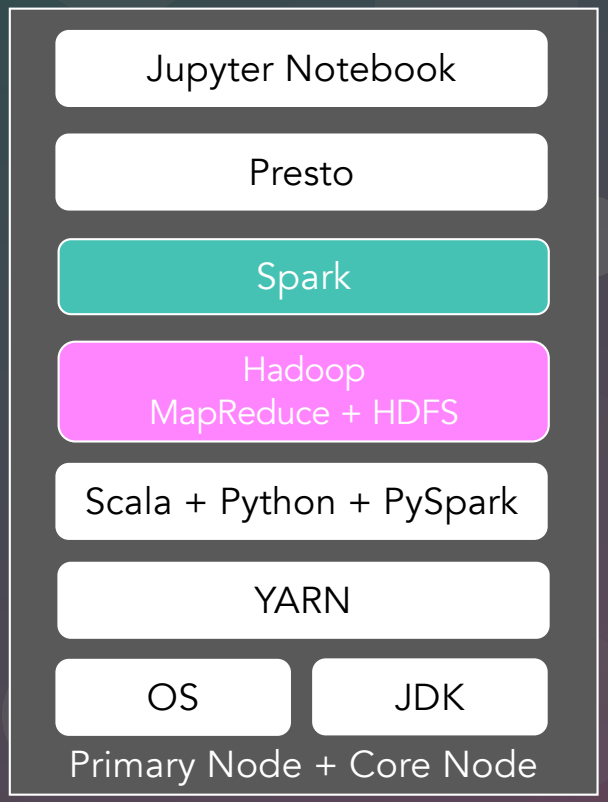
EMR Architecture

- Primary Node
- Core Node
- Task Node



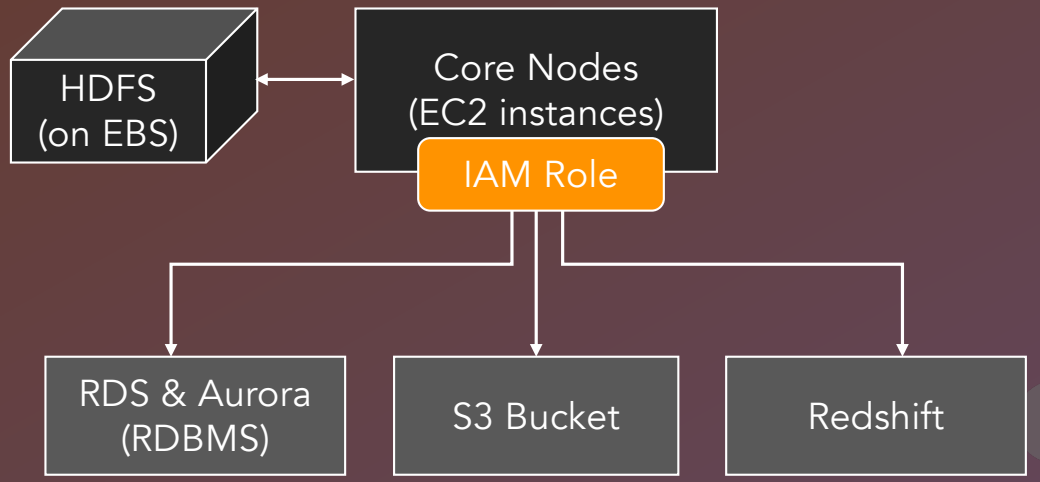
EMR Components

- Applications (Spark, Hive, Presto, Hadoop etc.).
- EC2 instance types for Primary, Core & Task nodes.
- EBS volume size.
- VPC, subnet, security groups.
- Software Settings.
- Steps.
- EC2 key pair.
- IAM – EMR Service Role, EC2 instance profile role.
- Applications – Spark, Hadoop, Presto, Jupyter.



AWS Service Communication

- EMR has to access multiple AWS services.
  - RDS or Aurora
  - Redshift
  - Snowflake
  - DynamoDB
- EMR need to access these services - using IAM service roles.
- RDS/Aurora – Database read & write.
- Redshift – DWH read & Write.



- S3 – i) Spark History Logs; ii) PySpark Code; iii) File sources & targets.

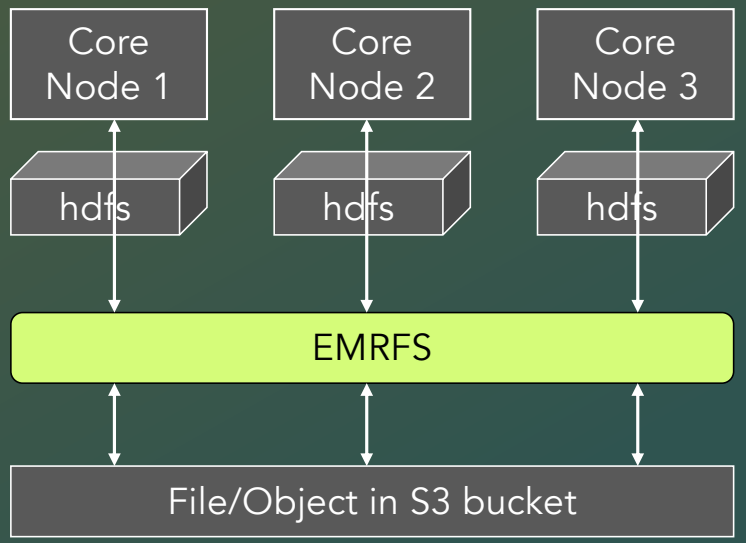
spark-defaults.conf & spark-env.sh

- Spark configuration parameters.
- Spark Env file settings.
- EMR Software settings.

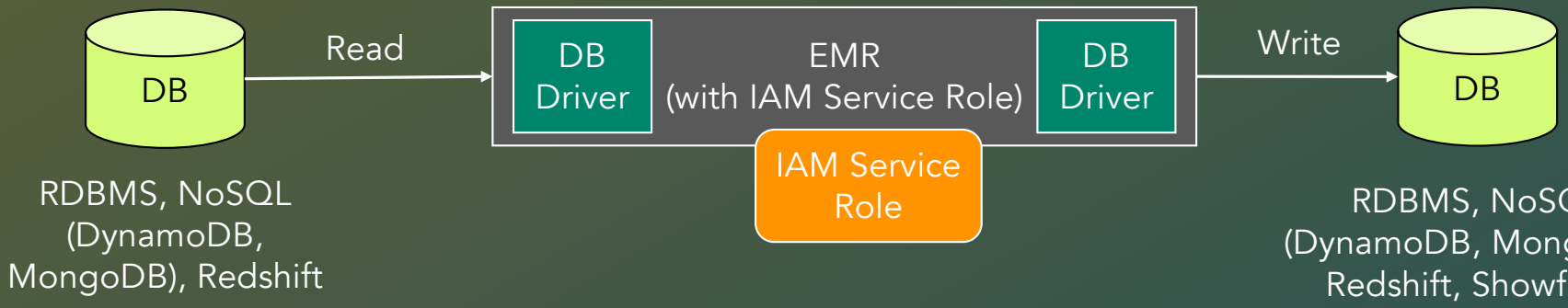
```
[
  {
    "Classification": "spark-defaults",
    "Properties": {
        "spark.default.parallelism" : "16",
         "spark.driver.memory" : "4g",
         "spark.executor.memory" : "4g",
         "spark.executor.cores" : "4"
    }
  },
  {
    "Classification": "spark-env",
    "Properties": {},
    "Configurations" : [
          "Classification": "export",
          "Properties": {
                   "PATH": "</path>"
          }
    ]
  }
]
```

File Sources & Targets

- HDFS on EBS. Access using hdfs://
- HDFS is backed by EBS instance store. It is temporary in nature.
- Terminating cluster will remove data from EBS.
- HDFS provides low latency access.
- For permanent store S3 can be used via EMRFS. High latency access.
- Access using S3 URI in PySpark Code.
- For low latency, copy files from S3 to HDFS.



DB Sources & Targets



- Database drivers need to be available. Most of the DB drivers are already available on EMR.
- IAM service role should have access to AWS & non-AWS database services.

EMR & Java

- EMR version 7.x uses Java version 17.
- EMR version 6.x uses Java version 8.

YARN on EMR

- YARN components are automatically installed on EMR.
- Resource Manager.
- Node Manager.
- Containers.
- Configuration parameter values can be provided using yarn-site classification.
- Spark drivers and executors run in YARN containers.

```
[
  {
    "Classification": "yarn-site",
    "Properties": {
      "yarn.scheduler.maximum-allocation-mb": "4096",
      "yarn.scheduler.maximum-allocation-vcores": "4"
    }
  }
]
```

YARN on EMR

- yarn.scheduler.maximum-allocation-mb (8192) : Max memory allocation for YARN container.
- yarn.scheduler.minimum-allocation-mb (1024) : Min memory allocation for YARN container.
- yarn.scheduler.minimum-allocation-vcores (1) : Min CPU allocation for YARN container.
- yarn.scheduler.maximum-allocation-vcores (32) : Min CPU allocation for YARN container.
- yarn.nodemanager.resource.memory-mb (8192) : Memory that is allocated to a container.
- yarn.nodemanager.resource.cpu-vcores (8) : CPU allocated to a container.

Submit Spark Application on EMR

- Option 1 – spark-submit from EMR Primary Node.
  - Copy PySpark code from local machine to S3.
  - SSH into EMR Primary Node.
  - Use spark-submit from EMR Primary Node.
- Option 2 – Submit from local machine to EMR using Steps.
  - Copy PySpark code to S3.
  - Use AWS CLI to submit job on EMR cluster using EMR Steps.

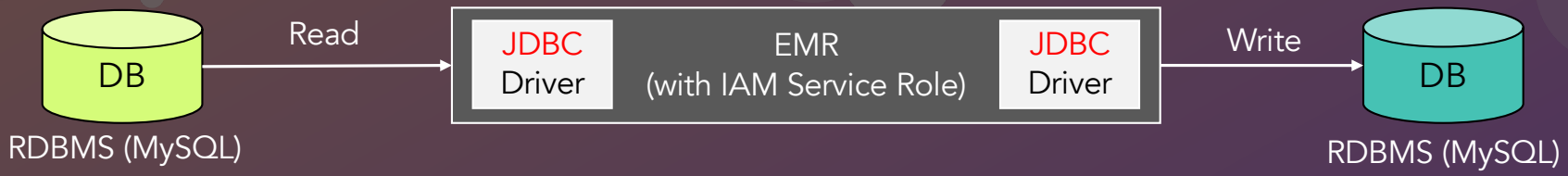
Spark UI

- Just like Laptop.

HandsOn – Running Spark on EMR

- Create EMR cluster 1 with following YARN & Spark Configuration.
  - YARN container size – 4 GB memory & 4 CPUs.
  - Spark default executor size – 6 GB memory & 6 CPUs.
- Clone the EMR cluster with following YARN & Spark Configuration.
  - YARN container size – 8 GB memory & 8 CPUs.
  - Spark default executor size – 4 GB memory & 4 CPUs.
- What is the difference between the two above?
- Experiment with S3 and HDFS:
  - Copy flight and finance datasets from laptop to S3.
  - Copy the same datasets from S3 to HDFS on EMR.
  - Execute Layover Analysis and Data Skewness applications on EMR from laptop.
  - Explore Spark UI. Execute the application above with files on S3 and HDFS. What is the time difference?

DB Sources & Targets



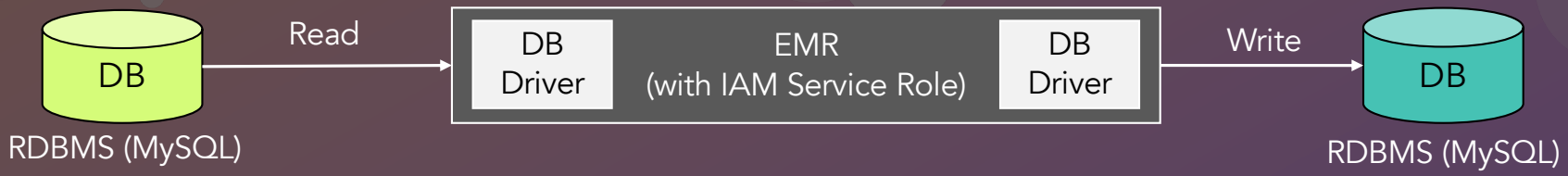

```
spark.read.format("jdbc")
  .option("url", "<url>")
  .option("dbtable", "<dbname>.<table-name>")
  .option("user", <user>)
  .option("password", <pwd>)
  .option("driver", "com.mysql.cj.jdbc.Driver")
  .option("query", "SELECT query")
  .option("partitionColumn", "<column>")
  .option("numPartitions", "10")
  .option("fetchSize", "1000000")
```

- Query – Provide custom SELECT statement to select rows. Dbtable option cannot be used.
- partitionColumn – Single column to partition table while reading from multiple worker nodes. Has to be Date, Timestamp, Numeric data type. lowerBound, upperBound, numPartitions must be specified.
- numPartitions – Max no. of partitions that can be used to read a table.
- fetchSize – No. of rows to fetch per connection trip.

partitionColumn, numPartitions, lowerBound, upperBound

- numPartitions – No. of partitions for parallel processing.
  - Read – Should be based on the no. of tasks or CPUs.
  - Write – Each partition/task would create one connection to the database to write.
- partitionColumn – Column based on which Spark would partition the rows for DataFrame.
- lowerBound – Min value of the partition column.
- upperBound – Max value of the partition column.
  - Example – partitionColumn = customer_id numPartitions = 100, lowerBound = 1000000, upperBound = 3000000
  - Total rows per partition = (3,000,000 – 1,000,000) / 100 = 10000

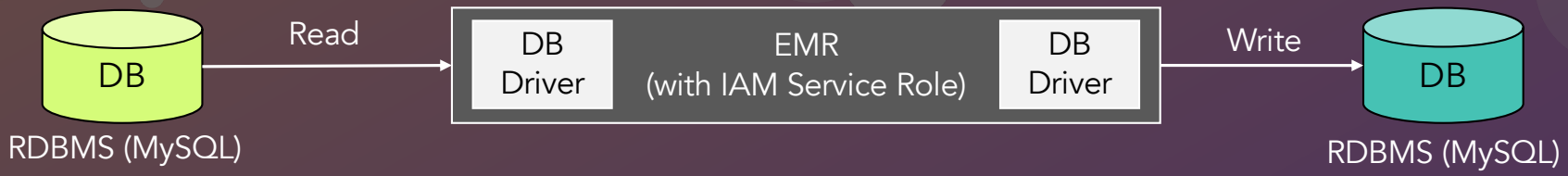
DB Sources & Targets




```
.....
  .option("pushDownPredicate", "true")
  .option("pushDownAggregate", "true")
  .option("pushDownLimit", "true")
  .load()
```

- pushDownPredicate – Push filter and where APIs to the database source as much as possible.
- pushDownAggregate – Push aggregate functions to database source. Only possible if the aggregate functions are supported at source.
- pushDownLimit – LIMIT is pushed down to database source.

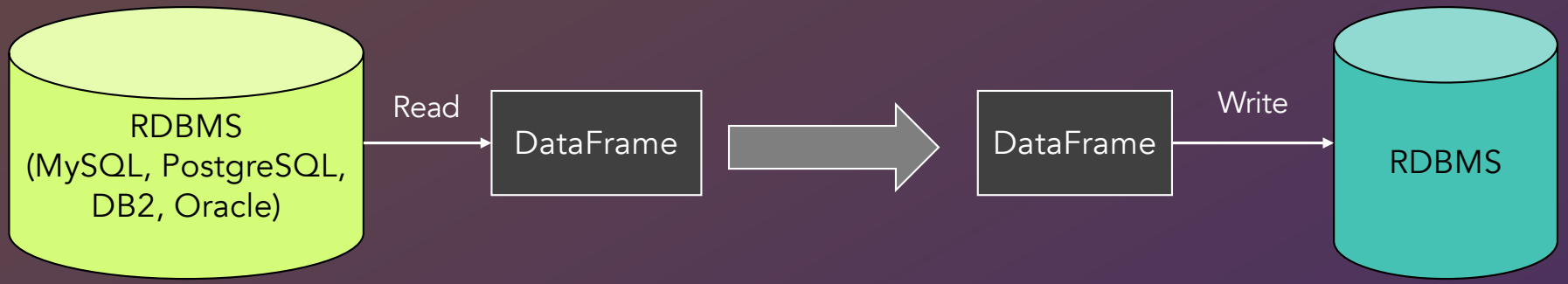
DB Sources & Targets




```
spark.write.format("jdbc")
  .option("url", url)
  .option("dbtable", "<dbname>.<table-name>")
  .option("user", <user>)
  .option("password", <pwd>)
  .option("driver", "com.mysql.cj.jdbc.Driver")
  .option("numPartitions", "10")
  .option("batchSize", "10000")
  .option("truncate", "true")
  .mode("append|overwrite|ignore|error")
  .save()
```

- numPartitions – Max no. of partitions that can be used to write to a table.
- bacthSize – No. of rows to insert per connection trip.
- truncate – Truncate the target table before loading/inserting. Applicable when mode in Overwrite.

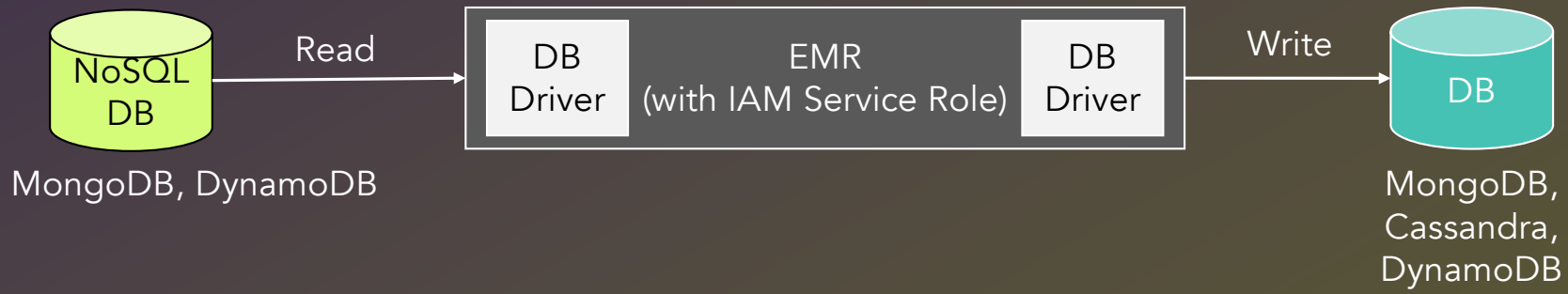
DB Sources & Targets



DWH Sources & Targets

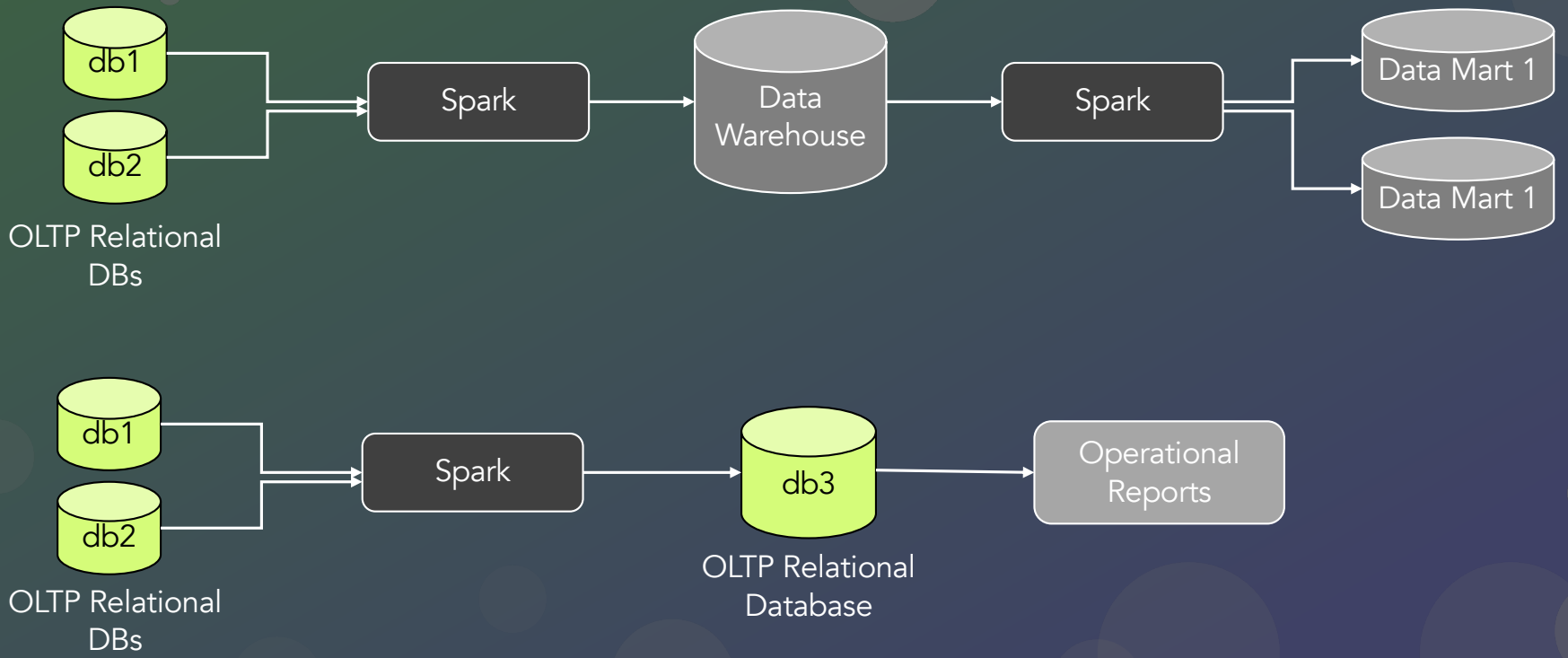

```
url = jdbc:redshift://<endpoint>:5439/<database>?user=<user>&password=<pwd>

sql_context.read
  .format("io.github.spark_redshift_community.spark.redshift")
  .option("url","<url>")
  .option("dbtable","<table-name>")
  .option("aws_iam_role","<iam-role-arn>")
  .option("tempdir","<s3-temp-folder>")
  .load()
  .mode("append|overwrite|ignore|error")
  .save()
```

NoSQL Sources & Targets

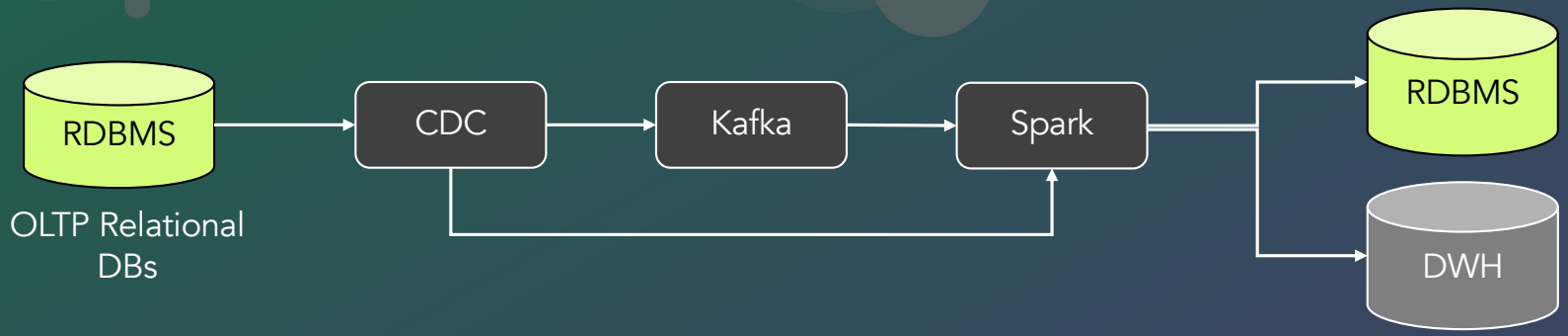


- JAR file has to be copied to S3 location and provide the location while submitting the job

DB Sources & Targets – Use Cases



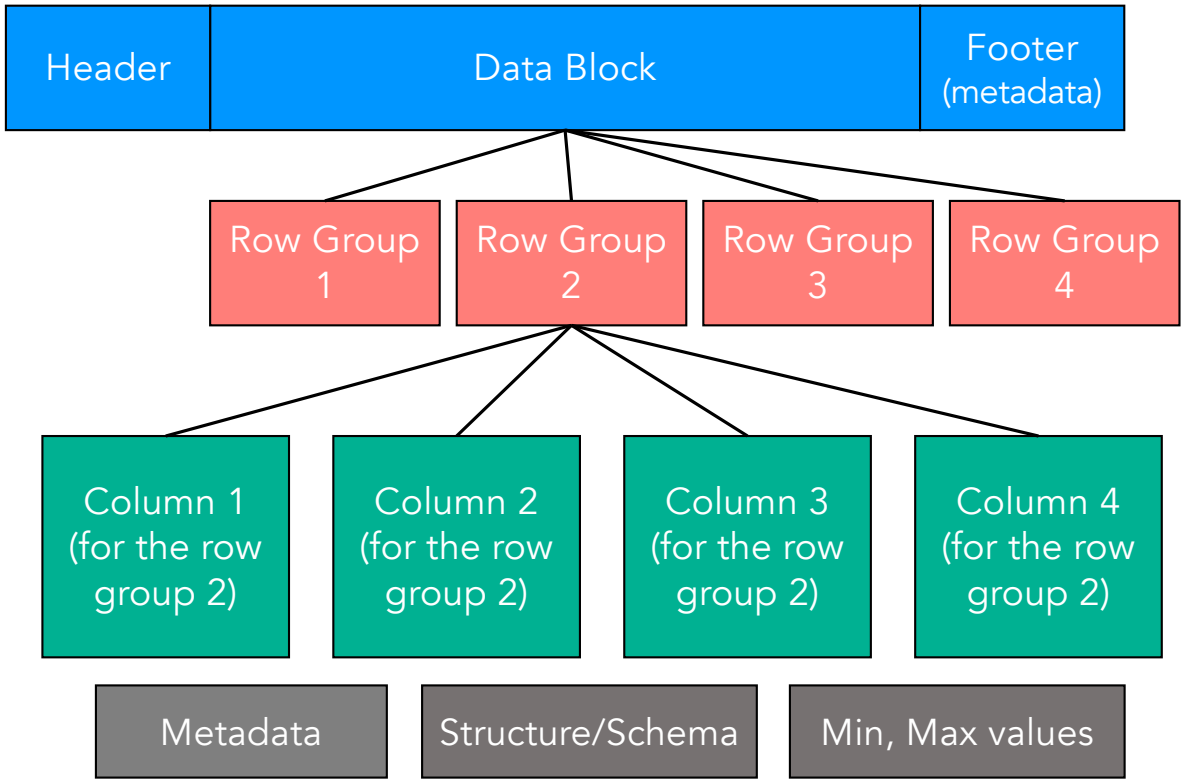
DB Sources & Targets – Use Cases



- Real-time fraud detection.
- Recommendation engine.
- Credit score analysis.
- Predictive analysis.
- Pro-active maintenance.
- Targeted marketing.
- Traffic route optimization.

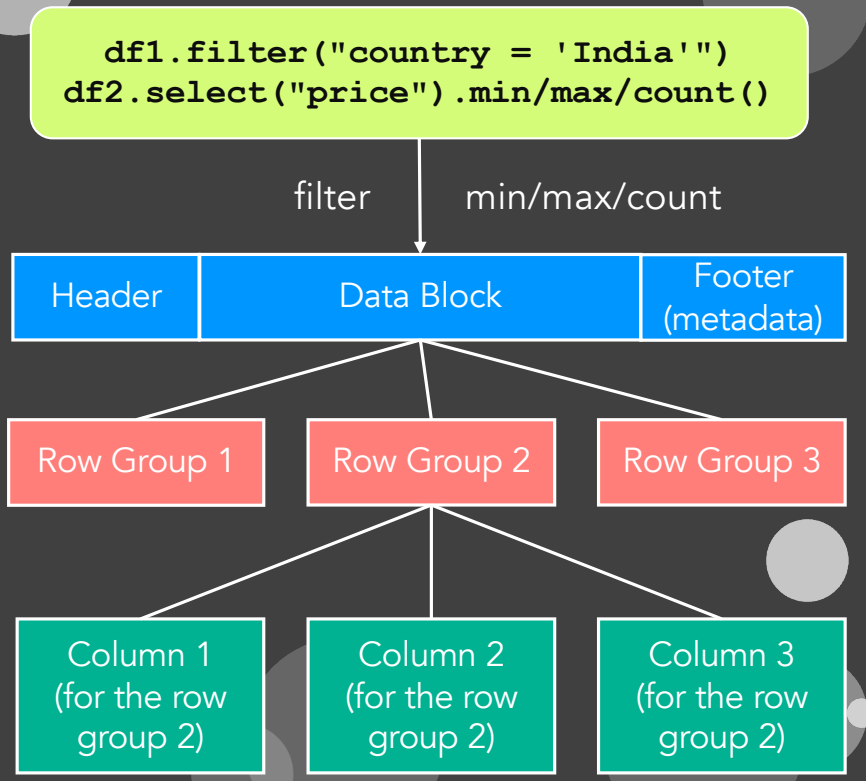
Parquet File Architecture

- Hierarchical.
- Header, Data Block, Footer.
- Row Group – Collection of a set of rows. Actual data block (128 MB).
- Metadata helps in reducing the no. of block reads.
- File metadata is used by readers to find the interested column chunks.
- Column chunks are read sequentially.



Parquet Files – Optimizations

- Predicate or Filter pushdown
  - spark.sql.parquet.filterPushdown
  - spark.sql.parquet.recordLevelFilter.enabled
- Aggregate pushdown
  - spark.sql.parquet.aggregatePushdown



Parquet Files – Schema Evolution

- Schema evolution - spark.sql.parquet.mergeSchema
  - Schema is merged from all files.

Handling JSON Files

- Sources:
  - NoSQL DBs – DynamoDB, MongoDB, CouchBase etc.
  - REST APIs.
- JSON data structure,
- Things to consider while working with JSON.
- Embedded JSON.
- JSON Array.
- explode, from_json, to_json, parse_json, schema_of_json.

```
{
  'passenger_id': 123456,
  'itinerary_no': 152976430,
  'passenger_details': {
     'name': 'Eulalia Alonso',
     'address': 'Spain'
  },
  'aircraft_ids': [
        152976430,
        150418396,
        159793105
  ],
  'travel_details': [
     {'flight_id':'STA42739', 'dep':'10:46:11', 'arr': '12:17:11'}
     {'flight_id':'STA42739', 'dep':'10:46:11', 'arr': '12:17:11'}
     {'flight_id':'STA42739', 'dep':'10:46:11', 'arr': '12:17:11'}
  ],
  'tickets' : ['JK-31565-LS', 'MJ-46690-MK', 'WZ-50521-SK']
}
```

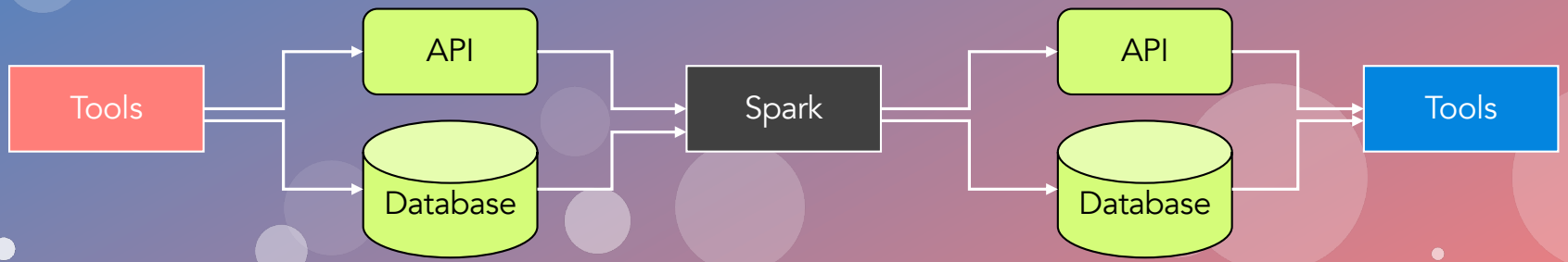
Embedded JSON

JSON Array

Embedded JSON in Array

Other Source & Targets

- Business Applications
- CRM – SalesForce, Microsoft Dynamics
- ERP – Workday, SAP
- Ticketing Tools – JIRA, ServiceNow
- Collaboration – Zoom, Slack, MS Teams
- Monitoring – Datadog, Grafana, Splunk, NewRelic



Summary

- EMR architecture recap
- Software Settings
- EMR Steps
- IAM role for EMR service
- EC2 instance profile role for Core Nodes
- Spark configuration on EMR
- YARN configuration
- Submit Spark application to EMR
- RDBMS source & target
- Redshift source & target
- Parquet file optimizations
- Handling JSON files